

Comparación de Estrategias de Predicción de Saltos

1. Introducción

El presente proyecto tiene como objetivo comparar diferentes estrategias de predicción de saltos utilizadas en arquitectura de computadores. Las estrategias evaluadas son: **Predict Not Taken**, **Predict Taken** y **Predictor de 2 Bits**.

La predicción de saltos es una técnica importante para mejorar el rendimiento de los procesadores, especialmente en arquitecturas que utilizan **pipelining**. Cuando un procesador encuentra una instrucción de salto condicional, no sabe inmediatamente cuál será la siguiente instrucción que debe ejecutar. Por esta razón, intenta predecir si el salto será tomado o no tomado.

En este proyecto se implementa una simulación en Python que permite analizar el comportamiento de cada predictor usando diferentes trazas de instrucciones de salto.

2. Relación con el tema del curso

El proyecto se relaciona con la Unidad 5 del curso, titulada **“Mejorando el Desempeño”**. En esta unidad se explican conceptos como **Instruction Level Parallelism**, **pipelining** y **riesgos del pipeline**.

El pipelining permite dividir la ejecución de una instrucción en varias etapas, como búsqueda, decodificación, ejecución y escritura. Esto mejora el rendimiento porque varias instrucciones pueden procesarse al mismo tiempo en diferentes etapas.

Sin embargo, aparecen problemas llamados **hazards** o riesgos del pipeline. Uno de ellos es el **riesgo de control**, que ocurre cuando una instrucción de salto impide saber cuál será la siguiente instrucción que debe cargarse. La predicción de saltos busca reducir este problema anticipando el resultado del salto.

3. Objetivo del proyecto

El objetivo principal del proyecto es comparar tres estrategias de predicción de saltos y medir su rendimiento mediante la **tasa de aciertos**.

Los objetivos específicos son:

- Simular trazas de instrucciones de salto.
- Implementar predictores estáticos y dinámicos.
- Comparar los resultados de cada predictor.
- Calcular la cantidad de aciertos, fallos y porcentaje de precisión.

- Mostrar los resultados mediante una interfaz gráfica, tabla y gráfica comparativa.
-

4. Concepto de traza de saltos

Una traza de saltos es una secuencia que representa el resultado real de varias instrucciones de salto. En el proyecto se utilizan dos valores:

- **T** : salto tomado.
- **N** : salto no tomado.

Ejemplo de una traza:

```
T T N T N
```

Esto significa que el primer salto fue tomado, el segundo también, el tercero no fue tomado, el cuarto fue tomado y el quinto no fue tomado.

Las trazas permiten probar cómo se comporta cada predictor frente a diferentes patrones.

5. Predictores implementados

5.1 Predict Not Taken

El predictor **Predict Not Taken** siempre predice que el salto no se tomará. Es decir, siempre responde **N**.

Es una estrategia estática porque no aprende de los resultados anteriores. Su funcionamiento es simple, pero puede ser efectivo cuando la mayoría de los saltos reales son no tomados.

Ejemplo:

```
Traza real:      N N N T N
Predicción:      N N N N N
```

En este caso tendría varios aciertos porque predominan los valores **N**.

5.2 Predict Taken

El predictor **Predict Taken** siempre predice que el salto sí se tomará. Es decir, siempre responde **T**.

También es una estrategia estática, porque no cambia su comportamiento durante la ejecución. Funciona mejor cuando la mayoría de saltos reales son tomados.

Ejemplo:

```
Traza real:    T T T N T
Predicción:    T T T T T
```

En este caso tendría buen rendimiento porque la mayoría de valores reales son **T**.

5.3 Predictor de 2 Bits

El **Predictor de 2 Bits** es una estrategia dinámica. A diferencia de los dos anteriores, este predictor sí aprende del comportamiento anterior de la traza.

Utiliza cuatro estados:

Estado	Significado	Predicción
0	Fuertemente no tomado	N
1	Débilmente no tomado	N
2	Débilmente tomado	T
3	Fuertemente tomado	T

El predictor inicia en el estado 1, llamado **débilmente no tomado**. Si el estado es 0 o 1, predice **N**. Si el estado es 2 o 3, predice **T**.

Después de cada salto, el estado se actualiza:

- Si el resultado real fue **T**, el estado aumenta.
- Si el resultado real fue **N**, el estado disminuye.
- El estado mínimo es 0 y el máximo es 3.

Este predictor es útil porque puede adaptarse a patrones repetitivos. Sin embargo, puede fallar cuando la traza cambia constantemente, como en una secuencia alternada.

6. Estructura del proyecto

El proyecto está organizado en varias carpetas y archivos, cada uno con una responsabilidad específica.

6.1 `main.py`

Este archivo es el punto de entrada del proyecto. Su función principal es iniciar la interfaz gráfica.

Cuando se ejecuta:

```
python main.py
```

el programa llama a la función que abre la aplicación.

6.2 `gui/app.py`

Este archivo contiene la interfaz gráfica del sistema. Desde esta interfaz el usuario puede:

- Cargar una traza predefinida.
- Escribir una traza manual.
- Generar una traza aleatoria.
- Ejecutar el análisis.
- Ver los resultados en una tabla.
- Ver una gráfica comparativa.
- Exportar los resultados a Excel.
- Ejecutar una simulación grande con miles de instrucciones.

La interfaz no realiza directamente los cálculos de predicción. Su función es conectar al usuario con la lógica del sistema.

6.3 `utils/generator.py`

Este archivo se encarga de manejar las trazas. Contiene funciones para:

- Generar trazas aleatorias.
- Obtener trazas predefinidas.
- Analizar una traza.

Las trazas predefinidas incluidas en el proyecto son:

- Mayormente tomada.
- Mayormente no tomada.
- Alternada.
- Mixta.

Esto permite evaluar los predictores en diferentes escenarios.

6.4 `core/predictors.py`

Este archivo contiene la implementación de los tres predictores:

- `PredictorNotTaken`.
- `PredictorTaken`.
- `Predictor2Bits`.

Cada predictor tiene un método para realizar la predicción y otro para actualizarse después de conocer el resultado real.

En los predictores estáticos, la actualización no cambia nada. En el predictor de 2 bits, la actualización modifica el estado interno.

6.5 `core/evaluator.py`

Este archivo se encarga de evaluar el rendimiento de los predictores.

Su función principal es comparar la predicción de cada estrategia con el resultado real de la traza. Para cada instrucción de salto, el evaluador:

1. Solicita la predicción al predictor.
2. Compara la predicción con el resultado real.
3. Cuenta si fue acierto o fallo.
4. Actualiza el predictor.
5. Guarda el historial del proceso.

Al final, calcula la tasa de aciertos.

7. Fórmula de evaluación

La métrica principal utilizada en el proyecto es la **tasa de aciertos**.

La fórmula es:

$$\text{Tasa de aciertos} = (\text{aciertos} / \text{total de saltos}) \times 100$$

Por ejemplo, si un predictor acierta 8 veces en una traza de 10 saltos:

$$\text{Tasa de aciertos} = (8 / 10) \times 100 = 80\%$$

Esta fórmula permite comparar objetivamente cuál predictor tuvo mejor rendimiento.

8. Flujo de funcionamiento del sistema

El funcionamiento general del sistema es el siguiente:

1. El usuario abre el programa ejecutando `python main.py`.
2. La interfaz gráfica se carga.
3. El usuario selecciona o genera una traza.
4. La interfaz envía la traza al evaluador.
5. El evaluador ejecuta los tres predictores.
6. Cada predictor genera sus predicciones.
7. El sistema compara las predicciones con los resultados reales.
8. Se calculan aciertos, total de saltos y tasa de aciertos.
9. Los resultados se muestran en una tabla y en una gráfica.

Este flujo permite entender de forma visual y práctica cómo se comportan las estrategias de predicción.

9. Resultados esperados

Los resultados dependen del tipo de traza utilizada.

Si la traza es **mayormente tomada**, se espera que el predictor **Predict Taken** tenga mejor rendimiento, porque siempre predice `T`.

Si la traza es **mayormente no tomada**, se espera que el predictor **Predict Not Taken** tenga mejor rendimiento, porque siempre predice `N`.

Si la traza tiene un patrón estable, el **Predictor de 2 Bits** puede adaptarse y mejorar su tasa de aciertos.

Si la traza es alternada, por ejemplo:

T N T N T N

el predictor de 2 bits puede fallar, porque actualiza su estado con base en el resultado anterior, pero el patrón cambia constantemente.

10. Importancia del proyecto

Este proyecto es importante porque permite comprender de forma práctica un concepto usado en arquitectura de computadores.

La predicción de saltos ayuda a mejorar el rendimiento del procesador al reducir los efectos de los riesgos de control en el pipeline. Aunque el proyecto es una simulación simplificada, permite visualizar cómo diferentes estrategias pueden tener resultados distintos dependiendo del comportamiento de la traza.

Además, el proyecto permite comparar estrategias simples con una estrategia dinámica, mostrando que no siempre el predictor más avanzado gana, sino que su rendimiento depende del patrón de instrucciones.

11. Conclusiones

El proyecto demuestra que la predicción de saltos es una técnica útil para mejorar el rendimiento en procesadores con pipeline.

También se observa que no existe un predictor perfecto para todos los casos. El predictor **Predict Not Taken** funciona bien cuando predominan los saltos no tomados. El predictor **Predict Taken** funciona bien cuando predominan los saltos tomados. El **Predictor de 2 Bits** puede adaptarse a patrones repetitivos, pero puede fallar cuando la traza cambia constantemente.

En conclusión, este proyecto conecta la teoría del curso con una aplicación práctica en Python, permitiendo analizar y comparar el comportamiento de diferentes estrategias de predicción de saltos mediante resultados medibles y visuales.